1. Python Programs related to variables and expression showing impact of operators.

1) Arithmetic Operators a = 10 b = 3 print("Addition:", a + b) print("Subtraction:", a - b)

print("Multiplication:", a * b) print("Division:", a / b) print("Modulus:", a % b)

#OUTPUT:-

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Modulus: 1
```

2) Assignment Operators

x = 5

x += 3 # x = x + 3

print("After += :", x)

x *= 2 # x = x * 2

print("After *= :", x)

x -= 4 # x = x - 4

print("After -= :", x)

#OUTPUT:-

```
After += : 8
After *= : 16
After -= : 12
```

3) Logical

Operators x = True y

= False print("AND:",

x and y) print("OR:",

x or y) print("NOT x:",

not x) **#OUTPUT:-**

```
AND: False
OR: True
NOT x: False
```

4) Expression

Evaluation Example

a = 5 b = 2 c = 3

result = a + b * c print("Result:",

result)

#OUTPUT:-

```
Result: 11
```

2. Programs related to the conditional statements which shows different impact of If() and ladder if().

1) Using Multiple `if` num = 10 if num > 0:

print("Positive") if

num % 2 == 0:

print("Even")

#OUTPUT:-

```
Positive
Even
```

2) Using if-elif-else Ladder

num = 10 if num > 0:

print("Positive") elif

num % 2 == 0:

print("Even")

#OUTPUT:-

```
Positive
```

3. Programs related to the iterative statements which shows different impact of for() and nested for() and while() loop including break and continue statements.

1) for loop()

```
# Print numbers from 1 to 5 for

i in range(1, 6):

   print(i)
```

#OUTPUT:-

```
1
2
3
4
5
```

2)nested for loop() #

```
Print pattern for i in

range(1, 4):     for j in

range(1, 4):

print("*", end=" ")

print()
```

#OUTPUT:-

```
* * *
* * *
* * *
```

3)while loop()

```
# Print numbers from 1 to 5

i = 1 while i

<= 5:

   print(i)

i += 1
```

#OUTPUT:-

```
1
2
3
4
5
```

4) Break statement

```
# Stop loop when number is 3
```

```
for i in range(1, 6):

    if i == 3:

break     print(i)
```

#OUTPUT:-

```
1
2
```

5) Continue statement

```
# Skip number 3 for i

in range(1, 6):

    if i == 3:

continue     print(i)
```

#OUTPUT:-

```
1
2
4
5
```

4. Programs related to the string manipulation using conditional statements and iterations.

1) Check Vowels in a String string = input("Enter a string: ") count = 0 for ch in string: if

ch.lower() in 'aeiou':

count += 1 print("Number of

vowels:", count)

#OUTPUT:-

```
Enter a string: Hello Neha
Number of vowels: 4
```

2) Count digits, letters, spaces

Count different characters

string = input("Enter a string: ")

letters = digits = spaces = 0 for

ch in string: if ch.isalpha():

letters += 1 elif ch.isdigit():

digits += 1 elif ch.isspace():

spaces += 1 print("Letters:",

letters) print("Digits:", digits)

print("Spaces:", spaces)

#OUTPUT:-

```
Enter a string: Hello 12345 World
Letters: 10
Digits: 5
Spaces: 2
```

5. Programs related to the string built in functions and their uses in various manners.

1) upper() and lower() # Convert

string case text = input("Enter a

string: ") print("Uppercase:",

text.upper()) print("Lowercase:",

text.lower())

#OUTPUT:-

```
Enter a string: Hello Neha
Uppercase: HELLO NEHA
Lowercase: hello neha
```

2) strip()

Remove spaces text = input("Enter a

string with spaces: ") print("After strip:",

text.strip())

#OUTPUT:-

```
Enter a string with spaces:  Hello Neha
After strip: Hello Neha
```

3) replace()

Replace word text = input("Enter a

string: ") new_text =

text.replace("Python", "Java")

print("Updated string:", new_text)

#OUTPUT:-

```
Enter a string: Python is easy and Python is powerful.
Updated string: Java is easy and Java is powerful.
```

4) find()

Find position of substring text

= input("Enter a string: ") pos =

text.find("a") print("Position of

'a':", pos)

#OUTPUT:-

```
Enter a string: Python Programming
Position of 'a': 12
```

5) count()

Count occurrences text =

input("Enter a string: ") print("Count

of 'a':", text.count("a"))

#OUTPUT:-

```
Enter a string: Hello Neha Kiran Ghegade
Count of 'a': 3
```

6) split()

Split string into list text =

input("Enter a sentence: ")

words = text.split()

print("Words:", words)

#OUTPUT:-

```
Enter a sentence: Python is easy and powerful programming language
Words: ['Python', 'is', 'easy', 'and', 'powerful', 'programming', 'language']
```

7) len() # Find length text =

input("Enter a string: ")

print("Length:", len(text))

#OUTPUT:-

```
Enter a string: Hello Neha
Length: 10
```

6. Programs related to the function which implies the different effect of passing parameters and arguments.

1) positional vs keywords arguments def student(name, age): print("Name:", name) print("Age:", age) #

Positional student("Neha", 20) # Keyword student(age=20, name="Neha")

#OUTPUT:-

```
Name: Neha
Age: 20
Name: Neha
Age: 20
```

2) Immutable vs Mutable Argument

Immutable (int) def

change(x):

x = x + 5

print("Inside:", x)

num = 10

change(num)

print("Outside:", num)

Mutable (list) def

modify(lst):

lst.append(100)

numbers = [1, 2, 3]

modify(numbers)

print("List:", numbers)

#OUTPUT:-

```
Inside: 15
Outside: 10
List: [1, 2, 3, 100]
```

7. Programs related to the List, tuple, Set, and Dictionary which shows the impact on real implementation.

1) List marks = [80, 75, 90, 60] # Add new marks marks.append(85) # Update marks marks[1] = 78

print("Student Marks:", marks)

#OUTPUT:-

```
Student Marks: [80, 78, 90, 60, 85]
```

2) Tuple days = ("Mon", "Tue", "Wed",

"Thu", "Fri") print("Days:", days)

#OUTPUT:-

```
Days: ('Mon', 'Tue', 'Wed', 'Thu', 'Fri')
```

3) Set numbers = {1, 2, 3, 2, 1}

print("Unique numbers:", numbers)

numbers.add(5) print("After

adding:", numbers)

#OUTPUT:-

```
Unique numbers: {1, 2, 3}
After adding: {1, 2, 3, 5}
```

4) Dictionary student

= {

"name": "Neha",

"age": 20,

"marks": 85

}

# Access data
print("Name:", student["name"])

Update data student["marks"]

= 90 print("Updated:", student)

#OUTPUT:-

```
Name: Neha
Updated: {'name': 'Neha', 'age': 20, 'marks': 90}
```

8. Programs related to implementation of inheritance and polymorphism using various ways of class member and their accessibility.

ALL in One

```
class Animal:

    def __init__(self, name):

        self.name = name        # Public

        self._type = "Animal"   # Protected

        self.__sound = "Some sound"  # Private


    def speak(self):

        print(self.name, "makes", self.__sound)


class Dog(Animal):

    def __init__(self, name):

        super().__init__(name)

        self.__sound = "Bark"   # Private in child class


    def speak(self):  # Method overriding (Polymorphism)

        print(self.name, "says", self.__sound)


# Create objects

a = Animal("Generic Animal")

d = Dog("Tommy")


# Calling methods

a.speak()

d.speak()
```

```
•••  Generic Animal makes Some sound
     Tommy says Bark
```

1. Simple Inheritance Example

```
class Animal:

    def speak(self):

        print("Animal makes a sound")

class Dog(Animal):  # Inheriting from Animal

    def bark(self):

        print("Dog barks")

d = Dog()

d.speak()   # inherited method

d.bark()
```

```
Animal makes a sound
Dog barks
```

2. Method Overriding (Runtime Polymorphism)

```
class Animal:

    def speak(self):

        print("Animal speaks")

class Cat(Animal):

    def speak(self):  # overriding parent method

        print("Cat meows")

a = Animal()

c = Cat()

a.speak()

c.speak()   # different behavior (polymorphism)
```

```
Animal speaks
Cat meows
```

3. Method Overloading (Using Default Arguments)

```
class Math:

    def add(self, a, b=0, c=0):

        return a + b + c

m = Math()
```

```
print(m.add(5))       # 5

print(m.add(5, 10))    # 15

print(m.add(5, 10, 15))  # 30
```

```
•••   5
      15
      30
```

4. Access Specifiers (Public, Protected, Private)

```
class Student:

   def __init__(self):

      self.name = "John"       # Public

      self._age = 20          # Protected

      self.__marks = 90        # Private


 def show(self):

      print(self.name)

      print(self._age)

      print(self.__marks)


s = Student()

print(s.name)     # Public → accessible

print(s._age)     # Protected → accessible but discouraged

# print(s.__marks)  # Error (private)

s.show()  # Access private inside class
```

```
•••   John
      20
      John
      20
      90
```

5. Multilevel Inheritance

```
class A:

   def showA(self):

      print("Class A")
```

```
class B(A):
    def showB(self):
        print("Class B")


class C(B):
    def showC(self):
        print("Class C")


obj = C()
obj.showA()
obj.showB()
obj.showC()
```

```
Class A
Class B
Class C
```

9. Programs related to creation of class and objects which shows the implication of class member function and their accessibility.

```
class Student:

    def __init__(self, name, marks):

        self.name = name        # Public variable

        self._age = 18          # Protected variable

        self.__marks = marks    # Private variable


    # Public method

    def display(self):

        print("Name:", self.name)

        print("Age:", self._age)

        print("Marks:", self.__marks)


    # Private method

    def __grade(self):

        if self.__marks >= 50:

            return "Pass"

        else:

            return "Fail"


    # Public method to access private method

    def show_grade(self):

        print("Result:", self.__grade())



# Creating object

s1 = Student("Rahul", 75)


# Accessing public member

print(s1.name)
```

```
# Accessing protected member (possible but not recommended)

print(s1._age)


# Accessing private member directly → will give error

# print(s1.__marks)


# Access through public methods

s1.display()

s1.show_grade()
```

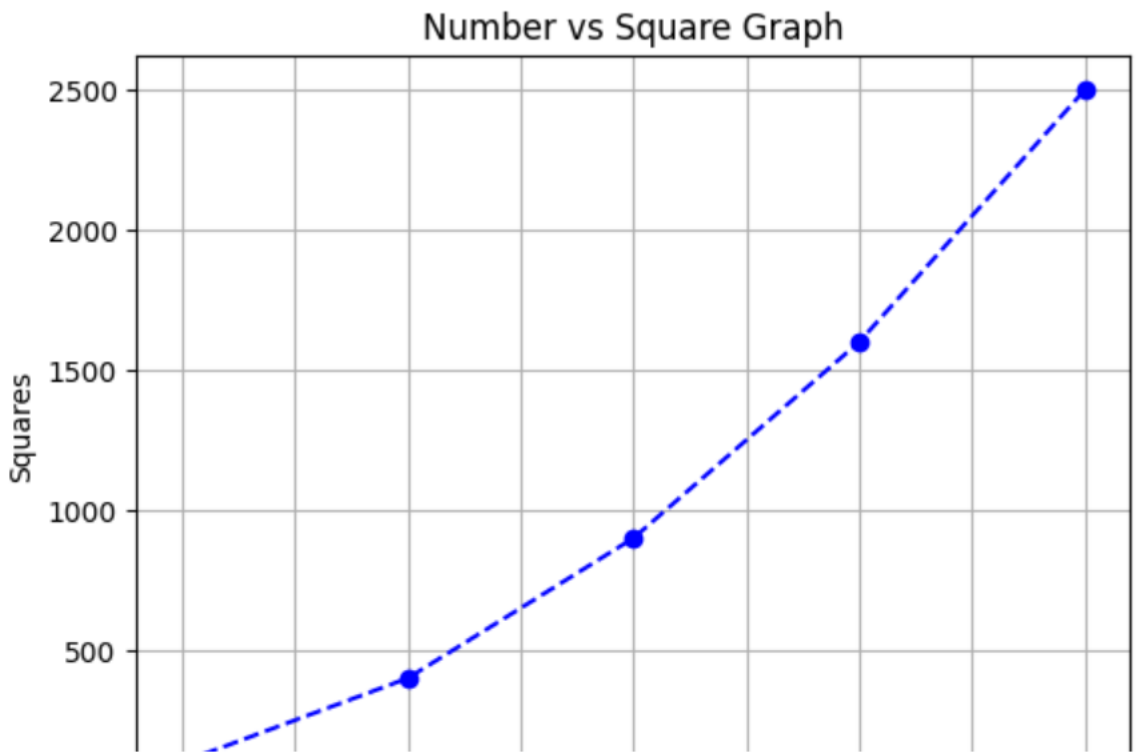
```
•••  Rahul
     18
     Name: Rahul
     Age: 18
     Marks: 75
     Result: Pass
```

10. Programs related programs using NumPy and least usable functions of NumPy like modification of chart, plot and labels to file handling to show the open, read, write and append execution.

```
import numpy as np

import matplotlib.pyplot as plt


# -------- NumPy Operations --------

data = np.array([10, 20, 30, 40, 50])

squared = data ** 2


print("Original Data:", data)

print("Squared Data:", squared)


# -------- Plot (Chart Modification) --------

plt.plot(data, squared, color='blue', marker='o', linestyle='--')

plt.title("Number vs Square Graph")

plt.xlabel("Numbers")

plt.ylabel("Squares")

plt.grid(True)


# Save chart to file

plt.savefig("chart.png")

plt.show()


# -------- File Handling --------


# Write to file

with open("data.txt", "w") as f:

    f.write("Original Data: " + str(data) + "\n")

    f.write("Squared Data: " + str(squared) + "\n")


# Append to file
```

```
with open("data.txt", "a") as f:

    f.write("Data processing completed.\n")


# Read from file

with open("data.txt", "r") as f:

    content = f.read()

    print("\nFile Content:\n", content)
```

```
Original Data: [10 20 30 40 50]
Squared Data: [ 100  400  900 1600 2500]
```



OR

10. Programs related Programs using NumPy and least usable functions of NumPy like modification of chart, plot, and labels. to file handling to show the open, read, write, and append execution.

1) NumPy + Plot import

numpy as np import

matplotlib.pyplot as plt x =

np.array([1, 2, 3, 4]) y =

np.array([10, 20, 30, 40])

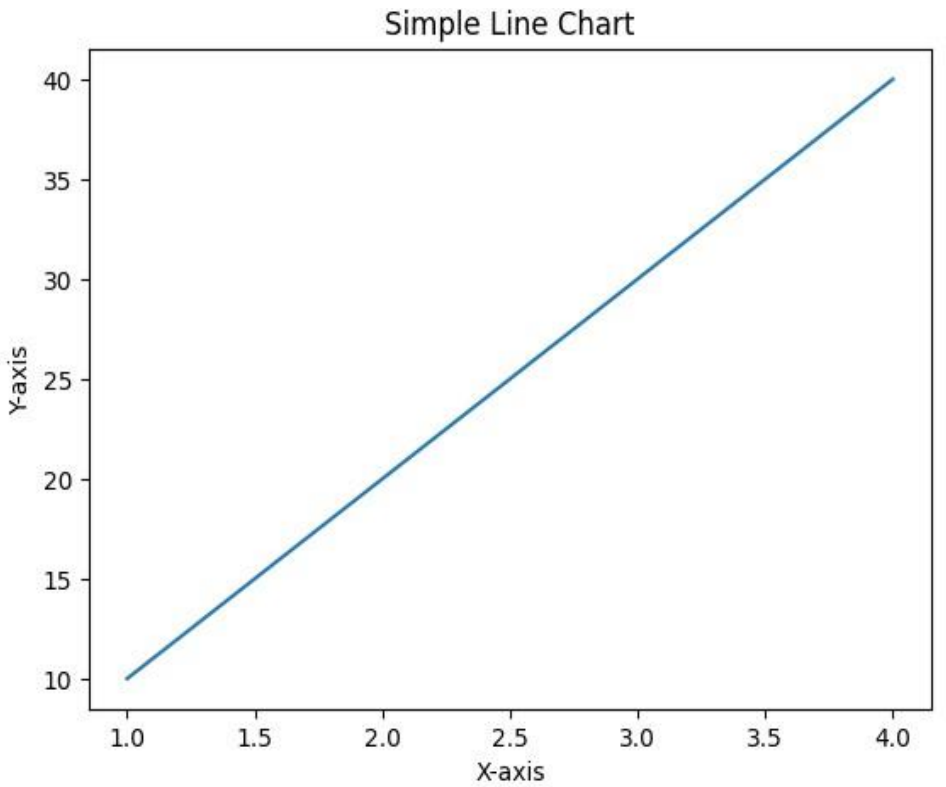
plt.plot(x, y) plt.title("Simple

Line Chart") plt.xlabel("X-

axis") plt.ylabel("Y-axis")

plt.show()

#OUTPUT:-



2) File Handling # Write to

file file = open("data.txt",

"w") file.write("Hello

Neha\n") file.close() #

Read file file =

open("data.txt", "r")

print(file.read())

file.close() # Append file

file = open("data.txt", "a")

file.write("Welcome!\n")

file.close()

#OUTPUT:-

```
Hello Neha
```

11. Programs related to indexing, slicing, sorting, and matrix manipulation.

1) Indexing and Slicing

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print("First element:", arr[0])

print("Slice (1 to 3):", arr[1:4])
```

#OUTPUT:-

```
First element: 10
Slice (1 to 3): [20 30 40]
```

2) Sorting

```
import numpy as np

arr = np.array([50, 10, 40, 20])

sorted_arr = np.sort(arr)

print("Sorted array:", sorted_arr)
```

#OUTPUT:-

```
Sorted array: [10 20 40 50]
```

3) Matrix Manipulation

```
import numpy as np

A = np.array([[1, 2], [3, 4]])

B = np.array([[5, 6], [7, 8]])

print("Addition:\n", A + B)

print("Multiplication:\n", A.dot(B))
```

#OUTPUT:-

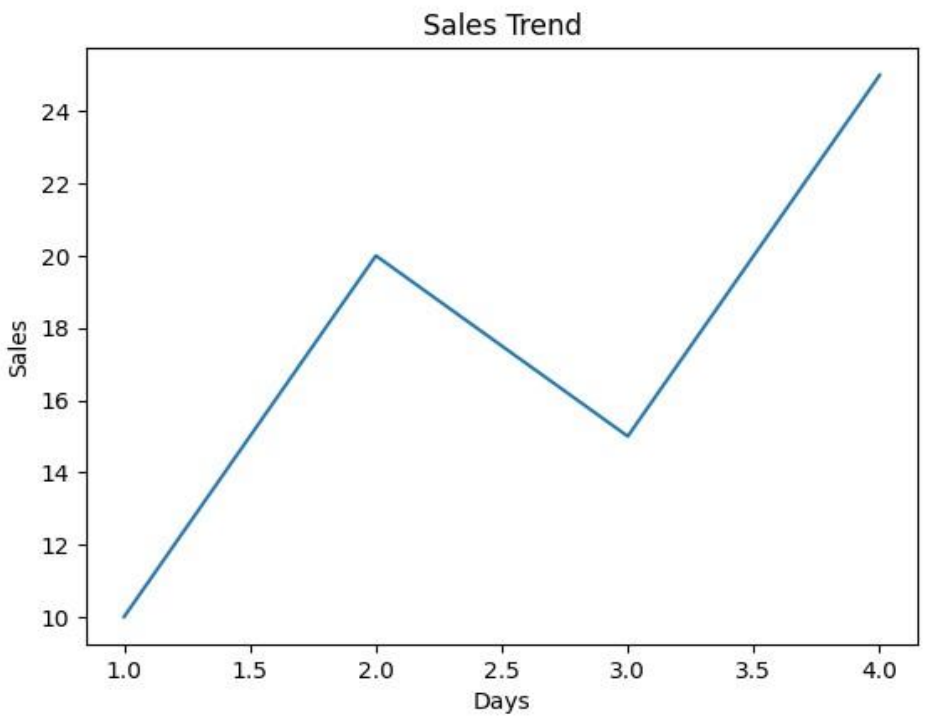
```
Addition:
 [[ 6  8]
 [10 12]]
Multiplication:
 [[19 22]
 [43 50]]
```

12. Programs using NumPy and least usable functions of NumPy like modification of chart, plot, and labels.

Chart with Labels

```
import numpy as np

import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 4])

y = np.array([10, 20, 15, 25])

plt.plot(x, y)

plt.title("Sales Trend")

plt.xlabel("Days")

plt.ylabel("Sales")

plt.show()
```

#OUTPUT:-



13. Programs using NumPy to show the classification and clustering of data accessing through storage file.

1) Basic Classification

```
import numpy as np

marks = np.array([35, 60, 80, 45, 90])

# Classification

result = []

for m in marks:

if m >= 50:

    result.append("Pass")

else:

    result.append("Fail") print(result)
```

#OUTPUT:-

```
['Fail', 'Pass', 'Pass', 'Fail', 'Pass']
```

2) Simple Clustering

```
import numpy as np

data = np.array([10, 12, 50, 55, 11, 52])

low = []

high = []

for d in data:

    if d < 30:

    low.append(d)

else:

    high.append(d)

print("Cluster 1:", low)

print("Cluster 2:", high)
```

#OUTPUT:-

```
Cluster 1: [np.int64(10), np.int64(12), np.int64(11)]
Cluster 2: [np.int64(50), np.int64(55), np.int64(52)]
```

14. Programs to use the Panda and related functions to show the

data analysis and their relations.

```
import pandas as pd

data = {
    "Name": ["Neha", "Riya", "Rahul", "Sneha", "Karan"],
    "Math": [85, 90, 78, 92, 88],
    "Science": [80, 95, 75, 89, 84],
    "English": [78, 85, 82, 88, 90]
}

df = pd.DataFrame(data)

print("Student Data:\n", df)

print("\nAverage Marks:\n", df.mean(numeric_only=True))
print("\nMaximum Marks:\n", df.max(numeric_only=True))

df["Total"] = df[["Math", "Science", "English"]].sum(axis=1)
print("\nData with Total:\n", df)

print("\nCorrelation between subjects:\n")
print(df[["Math", "Science", "English"]].corr())
```

```
...  Student Data:
          Name  Math  Science  English
     0    Neha    85       80       78
     1    Riya    90       95       85
     2   Rahul    78       75       82
     3   Sneha    92       89       88
     4   Karan    88       84       90

     Average Marks:
      Math       86.6
     Science    84.6
     English    84.6
     dtype: float64

     Maximum Marks:
      Math       92
     Science    95
     English    90
     dtype: int64

     Data with Total:
          Name  Math  Science  English  Total
     0    Neha    85       80       78    243
```

15. Programs to show the database connectivity and manipulation of the records like retrieving record, inserting record, deleting record, updating record, appending record.

```
import sqlite3

# -------- Connect to Database --------
conn = sqlite3.connect("student.db")
cursor = conn.cursor()

# -------- Create Table --------
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY,
    name TEXT,
    marks INTEGER
)
""")

# -------- Insert Records --------
cursor.execute("INSERT INTO students (name, marks) VALUES (?, ?)", ("Amit", 80))
cursor.execute("INSERT INTO students (name, marks) VALUES (?, ?)", ("Riya", 90))

conn.commit()

# -------- Append Record --------
cursor.execute("INSERT INTO students (name, marks) VALUES (?, ?)", ("Rahul", 75))
conn.commit()

# -------- Retrieve Records --------
print("All Records:")
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(row)

# -------- Update Record --------
cursor.execute("UPDATE students SET marks = 85 WHERE name = 'Amit'")
conn.commit()

# -------- Delete Record --------
cursor.execute("DELETE FROM students WHERE name = 'Rahul'")
conn.commit()

# -------- Display Updated Records --------
print("\nAfter Update and Delete:")
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
```

```
    print(row)

# -------- Close Connection --------
conn.close()
```

```
All Records:
(1, 'Amit', 85)
(2, 'Riya', 90)
(3, 'Neha', 80)
(4, 'Riya', 90)
(5, 'Neha', 80)
(6, 'Riya', 90)
(7, 'Neha', 80)
(8, 'Riya', 90)
(9, 'Amit', 85)
(10, 'Riya', 90)
(11, 'Amit', 80)
(12, 'Riya', 90)
(13, 'Rahul', 75)

After Update and Delete:
(1, 'Amit', 85)
(2, 'Riya', 90)
(3, 'Neha', 80)
(4, 'Riya', 90)
(5, 'Neha', 80)
(6, 'Riya', 90)
(7, 'Neha', 80)
(8, 'Riya', 90)
```

OR

```
import sqlite3

# Connect to database
con = sqlite3.connect("test.db")
cur = con.cursor()

# Create table
cur.execute("CREATE TABLE IF NOT EXISTS student (name TEXT, marks INTEGER)")

# Insert record
cur.execute("INSERT INTO student VALUES ('Amit', 80)")

# Append record (same as insert)
cur.execute("INSERT INTO student VALUES ('Riya', 90)")

# Retrieve records
cur.execute("SELECT * FROM student")
print("Records:", cur.fetchall())

# Update record
```

```
cur.execute("UPDATE student SET marks = 85
WHERE name = 'Amit'")

# Delete record
cur.execute("DELETE FROM student WHERE
name = 'Riya'")

con.commit()
con.close()
```

```
•••  Records: [('Amit', 80), ('Riya', 90)]
```